Incremental Data Drifting: Evaluation Metrics, Data Generation, and Approach Comparison

YU-TUNG PAI, CSIE, National Taiwan University, Taipei, Taiwan
NIEN-EN SUN, CSIE, National Taiwan University, Taipei, Taiwan
CHENG-TE LI, Department of Computer Science and Information Engineering, National Cheng Kung University, Tainan, Taiwan
SHOU-DE LIN, CSIE, National Taiwan University, Taipei, Taiwan

Incremental data drifting is a common problem when employing a machine-learning model in industrial applications. The underlying data distribution evolves gradually, e.g., users change their buying preferences on an E-commerce website over time. The problem needs to be addressed to obtain high performance. Right now, studies regarding incremental data drifting suffer from several issues. For one thing, there is a lack of clear-defined incremental drift datasets for examination. Existing efforts use either collected real datasets or synthetic datasets that show two obvious limitations. One is in particular when and of which type of drifts the distribution undergoes is unknown, and the other is that a simple synthesized dataset cannot reflect the complex representation we would normally face in the real world. For another, there lacks a well-defined protocol to evaluate a learner's knowledge transfer capability on an incremental drift dataset. To provide a holistic discussion on these issues, we create approaches to generate datasets with specific drift types, and define a novel protocol for evaluation. Besides, we investigate recent advances in the transfer learning field, including Domain Adaptation and Lifelong Learning, and examine how they perform in the presence of incremental data drifting. The results unfold the relationships among drift types, knowledge preservation, and learning approaches.

CCS Concepts: • **Information systems** → **Data mining**; • **Computing methodologies** → *Knowledge representation and reasoning;*

Additional Key Words and Phrases: Concept drift, incremental data drift, data generation

ACM Reference Format:
Yu-Tung Pai, Nien-En Sun, Cheng-Te Li, and Shou-de Lin. 2024. Incremental Data Drifting: Evaluation Metrics, Data Generation, and Approach Comparison. *ACM Trans. Intell. Syst. Technol.* 15, 4, Article 71 (July 2024), 26 pages. https://doi.org/10.1145/3655630

This work is supported by the National Science and Technology Council (NSTC) of Taiwan under grants 110-2221-E-006-136-MY3, 111-2221-E-002-146-MY3, 112-2628-E-006-012-MY3, 111-2221-E-006-001, and 112-2634-F-002-006.
Authors' Contact Information: Yu-Tung Pai, CSIE, National Taiwan University, Taipei, Taiwan; e-mail: r08944012@csie.ntu.edu.tw; Nien-En Sun, CSIE, National Taiwan University, Taipei, Taiwan; e-mail: r09922019@csie.ntu.edu.tw; Cheng-Te Li, Department of Computer Science and Information Engineering, National Cheng Kung University, Tainan, Taiwan; e-mail: reliefli@gmail.com; Shou-de Lin, CSIE, National Taiwan University, Taipei, Taiwan; e-mail: sdlin@csie.ntu.edu.tw.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 2157-6904/2024/07-ART71
https://doi.org/10.1145/3655630

1 INTRODUCTION

Machine learning (ML) has achieved tremendous success in solving various industrial problems including classification, recommendation, and recognition. In a standard supervised learning scenario, two assumptions are usually held: (1) training data comes all at once; (2) training data is of the same distribution as testing data. Therefore, a simple train-then-fix process for deploying a model can achieve decent performance. However, real-world applications, such as E-commerce recommendations, introduce complexities due to the sheer volume of data, such as user clicks and purchase histories, generated per second and the evolving nature of user behaviors over time. This dynamism and voluminous influx of data make the static model application infeasible and introduce the critical consideration of "Concept Drift."

Concept Drift, as articulated by [49], signifies the phenomenon wherein data distributions experience non-static, underlying shifts over time. The definition typically hinges on the joint distribution $P_t(X, y)$ between features X and labels y at a particular timestamp t [14, 30, 33]. To be more specific, given a dataset, in which data arrives at different timestamps, the concept drift happens if $\exists t : P_t(X, y) \neq P_{t+1}(X, y)$. Concept drift at time t can be defined as the change of joint probability of X and y at time t. Since the joint probability $P_t(X, y)$ can be decomposed into two parts: $P_t(X, y) = P_t(X) * P_t(y|X)$, three types of data drifting can be defined as below, also as illustrated in Figure 1.

- *Covariate Drift* [14] comes when $P(X)$ changes while $P(y|X)$ remains unchanged, i.e., $P_t(X) \neq P_{t+1}(X)$ and $P_t(y|X) = P_{t+1}(y|X)$. This happens when, due to some artifacts, the sampled data in each batch varies whereas the decision boundaries do not.
- *Actual Drift* [14] happens when, given two timestamps, $P(y|X)$ drifts while $P(X)$ stays the same, i.e., $P_t(X) = P_{t+1}(X)$ and $P_t(y|X) \neq P_{t+1}(y|X)$. An example is a situation where, when building a movie recommendation system, viewers may change their criteria to rate a comedy high as they watch more comedies.
- *Concept Drift* [14] indicates that where the changes of both $P(X)$ and $P(y|X)$ happen at the same time in two consecutive timestamps, i.e., $P_t(X) \neq P_{t+1}(X)$ and $P_t(y|X) \neq P_{t+1}(y|X)$. Either change occurs in the feature space or the mapping between features and labels can deteriorate the performance.

This work endeavors to facilitate learning amidst the aforementioned three types of data drifting in an incremental manner. *Incremental data drifting* [14] is indicative of the scenario where the data distribution *gradually* transitions over a specific duration. Such instances are not rare in real-world applications. Consider a scenario in a movie recommendation system: a user might gradually shift their preferences from action genres to documentaries as they explore various social and environmental issues. In a similar vein, envision the gradual increase in demand for electric cars as global awareness and advancements in sustainable technologies rise. Neither of these real-world evolutions occurred abruptly or without a certain trajectory. Rather, they unfolded progressively, highlighting the imperative of addressing incremental data drift in practical applications. Existing work on learning with concept drifts struggles to adequately accommodate incremental data drifting due to several reasons. First, there is a notable challenge in handling incremental drifting effectively for sequential data batches, as opposed to processing an instance-by-instance data stream. Second, a significant issue arises regarding how to precisely evaluate a model's performance amid the nuances of incremental data drifting. Third, the generation of incremental drifting datasets, which encompass covariate, actual, and concept types for astute model evaluation, presents a notable conundrum. Last, identifying the most effective approach to learning amidst incremental data drifting remains an unresolved inquiry. Below we discuss such four reasons in more detail.

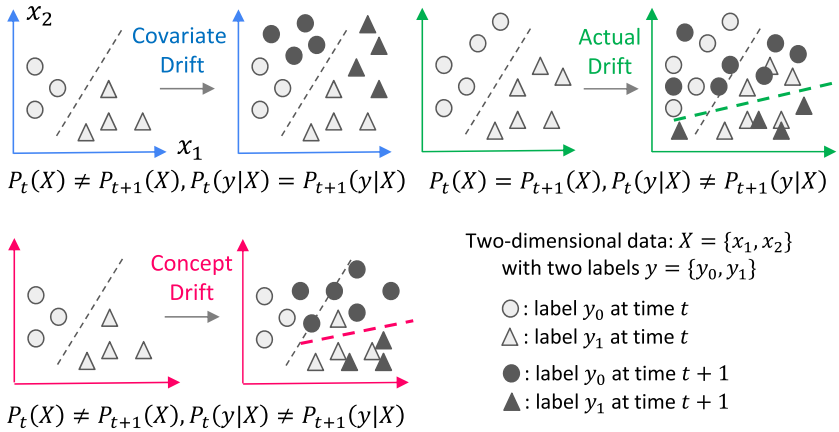

Fig. 1. Illustrations of three data drifting types.

- **Batch-wise Data.** Most of the methods tackling concept drift assume data arrives in the form of a data stream, where samples are examined one by one or in small chunks [30]. Therefore, these algorithms [10, 17] typically maintain a sliding window or a pool of previous models, and apply majority voting or monitor underlying drift through detection. However, we hold that this setting is potentially impractical in real-world applications for two reasons. For one thing, since users are continually generating data, the amount of data is increasing exponentially nowadays, which makes handling them instance by instance too slow. For another, collecting data with exact timestamps is expensive. Oftentimes, data comes without significant element-wise consequences but simply represents an overall outcome in a period of time, like buying preferences, or trends. As a result, we require an incremental drift learning framework that simulates a real ML-Pipeline where data arrives in *batches of great amount*.
- **Model Evaluation.** An effective learning model for incremental data drifting is expected to meet two requirements. One is not forgetting the data distribution it has seen so far. The other is being able to adapt to unseen data concerning incremental changes involved. *Backward Transfer* and *Forward Transfer* [29] are two well-known metrics to evaluate model learning over a continuum of data. We argue that such two metrics cannot precisely estimate the capability of transferring knowledge between incremental drifting data batches. The memorization of old data should be maintained throughout historical time, instead of only the oldest time or the latest time. Besides, both are not comparable with one another nor at the same scale because the performance scores at the time of random model initialization are in general low. We need proper evaluation metrics.
- **Data Generation.** Existing studies on concept drift suffer from a lack of clear-defined datasets for examinations. Wares et al. [48] point out a shortfall in the availability of public benchmark concept drift datasets. Besides, Lu et al. [30] also reveal the limitations of current concept drift datasets. In particular, for those real-world datasets that involve drifts, typically when and of which types the drifts happen are unclear. Without this information, evaluations are hard to make to conclude whether or not an algorithm can effectively handle drifts. On the other hand, for those synthetic concept-drift datasets, such as SINE [11], Rotation Hyperplane [46], and SEA [43], where instances are generated by artificially defined rules and settings, they are highly dependent on user-specified parameters. Different parameters lead to various results, which make direct comparisons difficult. In addition, being low in dimension and pure in shape, synthetic datasets cannot reveal real data patterns we will face

in industrial conditions, such as image recognition and fraud detection. For example, SEA dataset [43] is designed with three attributes where only two are relevant to drifts. If the sum of the two attributes exceeds a threshold, an instance is labeled 0, and vice versa. Drifts are created by changing the threshold through time. We aim to generate concept drift datasets of different types in an incremental fashion that represents real-world scenarios better.

— **Learning Approach.** Given the scenario that incremental drifting data arrives in a *batch-wise* manner, it is impracticable to learn based on existing approaches that handle data stream with concept drift [10, 17]. Nevertheless, if we consider batches with different data distributions as different domains, domain adaptation techniques [7] can be used. The incremental distribution changes in batches over time also fit the scenario of lifelong learning [8]. Domain Adaptation deals with domain shift by aligning latent spaces together while Lifelong Learning puts emphasis on not forgetting previously seen concepts. Although Domain Adaptation and Lifelong Learning target different schemas, they both embody solutions to handling discrepancies in data. To the best of our knowledge, none of past work has discussed methods in these fields on concept drift.

Note that addressing data management in the context of large volumes, this work juxtaposes the concept of "batch-wise data," entailing the handling of extensive datasets potentially in the tens of thousands of data points, with the utilization of a "small chunk" of data, often limited to a few hundred points [20, 31]. Our focus leans towards batch-wise data approaches, given their alignment with practical large-scale data management and computational efficiency in real-world scenarios. While employing small chunks for model updates offers a nimble approach, it may falter in scalability and robustness, especially in maintaining pace with the rapid influx of extensive data batches, potentially compromising the model's capability to swiftly adapt to varied patterns and comprehensive information encapsulated in larger datasets.

In this work, we highlight the learning problem of data drifting from two aspects: incremental fashion and batch-wise data sequence, which are essential, and practical, but hardly discussed in the literature. We term such kind of task as *incremental data drifting*, which is tightly coupled with covariate, actual, and concept drifts. To deal with the issues mentioned above, we first present two new metrics, *Old Transfer* and *New Transfer*, which can properly evaluate how a model memorizes the historical data distributions, and how a model adapts to future data with incremental changes, respectively. Second, we generate synthetic image and tabular datasets with *explicit* types of incremental drifting, i.e., covariate, actual, and concept, in a batch-wise setting. Third, we investigate how recent advances in domain adaptation and lifelong learning can be utilized to learn in the context of incremental data drifting.

Below we summarize the contributions of this work.

— We learn with incremental data drifting in a holistic view, including the batch-wise sequence data, defining evaluation metrics, generating the datasets, and examining various learning approaches. To the best of our knowledge, this is the first work that comprehensively looks into incremental data drifting.

— We propose two novel evaluation metrics, Old Transfer and New Transfer, to quantify the goodness of a learning model designed for incremental data drifting. Such two metrics quantify how well-drifting knowledge can be memorized by new models and simultaneously be adapted from old models.

— We generate synthetic incremental drifting datasets[1] with explicit drift types (i.e., covariate, actual, and concept) from existing image and tabular datasets. The data generation process

[1]Datasets are available at https://github.com/cealia/drift_dataset

itself, based on feature values and generative adversarial models, can be applied to more real datasets with similar properties.
— We identify that the approaches of domain adaptation and lifelong learning properly fit and tackle incremental data drifting. Extensive experiments conducted on both the generated and the real datasets deliver insights that unfold the relationships among drifting types, knowledge preservation, and learning approaches.

We organize this paper as follows. Section 2 reviews relevant studies. We present how to robustly evaluate a model that deals with incremental data drifting in Section 3, and describe how to generate synthetic incremental drifting datasets in Section 4. We give three approaches that can handle the learning with incremental data drifting in Section 5. The experimental results are reported in Section 6. We conclude this work in Section 7.

2 RELATED WORK

We review the relevant studies from four aspects. We first describe the typical methods for learning with concept drifts, then discuss existing concept-drift datasets. We also discuss recent advances in domain adaptation, lifelong learning, and out-of-distribution generalization that can deal with knowledge transfer between domains with different data distributions.

Solutions to Concept Drift. The **Drift Detection Method (DDM)** [13] keeps track of the online error rate within the time window, along with a pre-defined warning level and a drift level. If the error increases reaching the warning level, which implies the concept drift happens, a new model is built to learn the subsequent instances. If the error increases reaching the drift level, the old model is replaced by the new model for predictions. The **Early Drift Detection Method (EDDM)** [2] improves DDM using the distance between classification errors to detect concept drifts. Although these methods can adapt to the newest concepts, they may forget old concepts that are still consistent with new data. Learn++.NSE [10] constructs a pool of ensemble classifiers trained on different data batches. Weighted majority voting is applied to produce the final prediction. **Kappa Updated Ensemble (KUE)** [5] is also an ensemble-based approach that uses dynamic weighting and selection of base classifiers. A new classifier is added to the ensemble only when it has a positive contribution to improving the performance.

Addressing incremental data drifting demands a strategy that balances adapting to new concepts and preserving historical knowledge, a feat not fully realized by discussed methods. These predominantly employ binary, threshold-driven decisions for model retention or replacement, potentially sacrificing insights from older models and data. Incremental drifts, which may not significantly impact error rates or model performance immediately, could thereby escape detection and intervention by such approaches. While these methods exhibit a commendable rapid adaptation to new concepts, they often lack sturdy mechanisms for retaining valuable knowledge from historical data, presenting an ongoing challenge in learning amidst concept drift, particularly in the subtleties of incremental data drifting. The entwining of fresh and established knowledge, embodying both learning stability and plasticity, stands as a crucial area warranting further exploration and innovation in this domain.

Concept Drift Datasets. Several synthetic datasets and real-world datasets are widely used to evaluate the performance of an algorithm dealing with concept drift. Synthetic datasets such as SINE [11], Rotation Hyperplane [46], and SEA [43] are generated by user-specified parameters. Instances in these datasets are usually low in dimension, making it hard to reflect the concept drift phenomena in real industries. Although many real-world datasets, such as Email_data [23], Spam_data [23], and Gas Sensor Array Drift Dataset [45], have been created to tackle the above-mentioned issues, it is still unclear to know which types of drifts happen. Without this information,

one cannot conclude if an algorithm is effective to overcome a certain type of concept drift. In this work, we aim at generating new concept drift datasets that have the benefits of both existing synthetic and real-world datasets. Our compiled datasets can reflect real-world scenarios due to the high dimensionality of features, have drifting behaviors that happened in an incremental fashion, and can be utilized to measure the performance of a specific drift type for a given method.

Domain Adaptation. Given two data domains (i.e., source and target) drawn from different distributions, domain adaptation aims at effectively adapting the learning to target data by using only labeled source data and unlabeled target data. Most of the existing methods are to align the distributions of source and target domains through a feature extractor. DANN [16] creates an additional domain discriminator and a gradient reversal layer to force the feature extractor to align both domains. MCD [36] considers not only the alignment between domains, but also the task-specific decision boundaries between classes. DIRT-T [40] combines DANN with the cluster assumption, which means that the target data should be far from the decision boundary. **Adaptive Risk Minimization (ARM)** [51] is introduced in the context of domain generalization, where training data is structured into domains, and there might be multiple test-time shifts corresponding to new domains or domain distributions. While traditional approaches focus on learning a single robust model or invariant feature space that performs well across all domains, ARM takes a different approach. It aims to learn models that can adapt during test time to domain shifts using unlabeled test points. The central idea behind ARM is to optimize models for effective adaptation to shifts by learning to adapt based on the training domains. **Sequential Model Adaptation Using Internal distribution (SMAUI)** [35] is an algorithm that focuses on the learning of a parametric internal distribution, derived from the source domain, all within a unified embedding space. By harnessing the power of this internally sculpted distribution, SMAUI facilitates the alignment of source and target domain distributions. The adaptation to perform optimally in the target domain is achieved by sampling from this calculated internal distribution and compelling the target domain to adhere to a similar distribution in the embedding space. This is implemented through the minimization of the distance between the respective distributions, ensuring a smooth and effective adaptation of the model across various domains.

Existing domain adaptation methods like DANN, MCD, and SMAUI have shown proficiency in reconciling discrepancies between distinct data domains, yet their efficacy dwindles when confronted with the subtle and persistent nature of incremental data drifting. The core limitations stem from their often static and instantaneous adaptation mechanisms, which, while aptly managing abrupt or discrete domain shifts, inadequately address the slow, continuous evolution inherent to incremental drifts. Specifically, these methods tend to prioritize immediate adaptation and alignment between source and target distributions, potentially overlooking the cumulative impact of minute, ongoing changes in data distributions. Furthermore, their limited capacity to retain and utilize knowledge across varying phases of data evolution curtails their ability to generate predictions that are cogently aware of the entirety of data's temporal trajectory. Therefore, there is a pronounced need for approaches that not only adeptly adapt to immediate distributional disparities but also preserve and leverage historical data knowledge, ensuring nuanced, temporally-informed predictive performance amidst the gradual undulations of incremental data drifting. Nevertheless, we consider some domain adaptation methods useful when handling feature discrepancies in newcoming data (i.e., new concepts with covariate drift), and will have them experimentally compared to other approaches.

Lifelong Learning. Lifelong learning aims to mitigate the catastrophic forgetting of a learner, which means forgetting the knowledge learned from previous tasks after training on a new task with different data distribution. Various approaches are proposed: regularization-based approach, data rehearsal, generative rehearsal, and additional neural resource allocation. The

regularization-based approach [25] penalizes more on important parameters trained on previous tasks, and penalizes less on unimportant parameters meaning that they can be updated more easily when training on a future task. The data rehearsal approach [34] stores a limited amount of historical data utilized in old tasks, and adaptively reintroduces them at the training phase of new tasks. The generative rehearsal approach [39] trains a generator to produce old data instead of storing them. **Gradient-based Coreset Selection (GCR)** [44] is a replay-based continual learning framework. GCR employs gradient approximation as a strategic optimization criterion for coreset selection, astutely amalgamating recent progress within supervised learning environments. Ingeniously interwoven into the continual learning process, GCR prioritizes the selection and updating of replay buffers for ensuing training phases. Moreover, GCR judiciously incorporates a supervised representation learning loss into the continual learning objective, thereby enriching the representations learned throughout the model's lifecycle, and fostering a more robust and adaptive learning paradigm. **Trust Region Gradient Projection (TRGP)** [28] navigates forward knowledge transfer with an astute, layer-wise approach, introducing a 'trust region' to singularly select relevant old tasks for new ones using gradient projection norms mapped onto input subspaces. Recycling frozen weights from selected old tasks via a layer-wise scaling matrix, and concurrently optimizing scaling matrices and the model in directions orthogonal to old task subspaces, TRGP adeptly facilitates knowledge transfer while sidestepping forgetting, thereby judiciously balancing recall and adaptability in continual learning scenarios.

While current lifelong learning strategies like regularization-based, data rehearsal, and generative rehearsal approaches, alongside models like GCR and TRGP, offer innovative solutions to the catastrophic forgetting dilemma, their application in scenarios of incremental data drifting presents notable challenges. One such challenge stems from the meticulous balance these models strive to maintain between retaining knowledge from prior tasks and adapting to new ones. Given the subtlety and gradual progression of incremental drifts, existing methods might struggle to discern and adequately respond to slowly morphing data distributions, potentially misjudging the relevance and applicability of historical data and knowledge. Notably, the slow, nuanced nature of incremental data drifting might not sufficiently trigger adaptive responses in these models, as the gradual shifts may not introduce abrupt, discernible performance deteriorations. Furthermore, the models might not be able to differentiate between the necessity to retain previously learned knowledge and the imperative to adapt to minor alterations in data properties. This intricacy becomes especially pertinent when past concepts continue to hold relevance. Since we can consider learning from historical data as old tasks and prediction of new data as the new task, along with different data distribution, respectively, lifelong learning methods can be utilized to model incremental drifting between data batches. We will incorporate typical lifelong learning methods to examine how they perform in the context of incremental data drifting.

Out-of-distribution Generalization. Out-of-distribution (OOD) generalization refers to a model's ability to perform accurately on data that may come from a distribution different from the training data. This concept is pivotal in applications where models are deployed in dynamic and diverse real-world scenarios. **Invariant Risk Minimization (IRM)** [1] is a paradigm that seeks to identify and leverage invariant correlations across various training distributions to facilitate OOD generalization. The method aims to learn a data representation where the optimal classifier is consistent across all training distributions, linking learned invariances to underlying causal structures. **Risk Extrapolation (REx)** [26] addresses distributional shifts by assuming variations across training domains are indicative of potential test-time variations, even those of more extreme magnitudes. REx and its variants exhibit a capacity to recover causal mechanisms and provide robustness against input distribution changes, offering a balance between robustness to causally induced distributional shifts and covariate shift. Guo et al. [19] critically evaluate

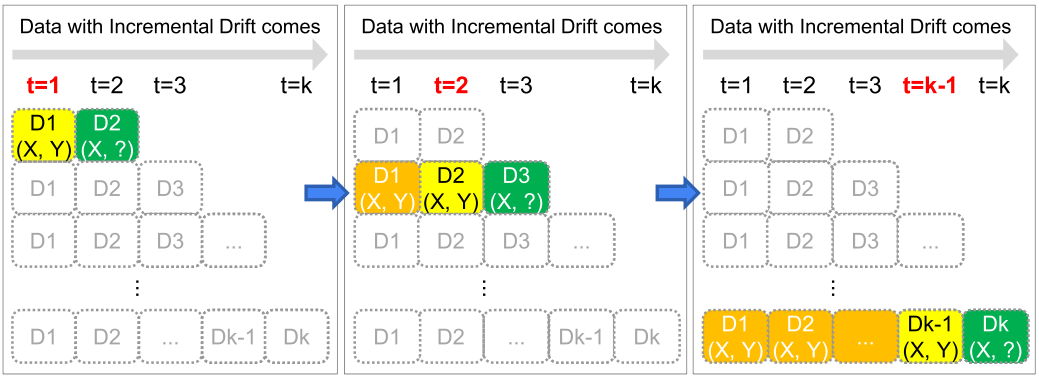

Fig. 2. Batches of drift data arrive sequentially. Yellow and orange blocks are at the current and historical time steps, respectively. Green ones are the next batches to be predicted.

IRM, particularly under conditions of strong spuriousness, where it tends to fail due to the robust spurious correlations. A solution is proposed by combining IRM with conditional distribution matching, mitigating specific types of spurious correlations under strong spuriousness. EIIL [6] is a framework for domain-invariant learning that infers partitions maximally informative for downstream Invariant Learning without explicit domain labels, establishing a connection between domain-invariant learning and algorithmic fairness.

The relationship between OOD generalization and incremental data drifting is nuanced. While OOD generalization focuses on ensuring model performance across diverse, unseen distributions, incremental data drifting pertains to the gradual, often subtle, shift in data distributions over time. Methods developed for OOD generalization, such as IRM [1] or REx [26], primarily aim to ensure robustness against stark, potentially abrupt distributional shifts and may not be directly applicable to scenarios of incremental data drifting due to their design and assumptions. Incremental data drifting requires models to continuously adapt and learn from the slowly changing data distribution, which is inherently a different problem from ensuring generalization across distinctly different distributions. Thus, while OOD generalization methods provide valuable insights into managing distribution shifts, they may not inherently cater to the subtleties and continuous adaptation required to handle incremental data drifting effectively.

3 EVALUATION FRAMEWORK

We consider a practical training protocol: (1) A large amount of data, which we refer to as a batch, arrives at a time. (2) Data in different batches drifts incrementally. (3) A learner can perform several passes over instances in a single batch. Figure 2 illustrates the logic of how batches of data are observed by a learner over time. Given the batch of the current time step (yellow) and the historical batches (orange) for model training, the goal is to make predictions on the data at the next time step (green).

Under this training protocol, there are two metrics that are critical to evaluating a learner's performance. The first is *Old Transfer*: the extent to which a learner is capable of not forgetting the data distribution it has seen so far. The second is *New Transfer*: how well the learner can adapt to unseen data concerning incremental changes involved. Regarding accessing a learner's ability to transfer knowledge, Lopez-Paz and Ranzato [29] have defined two types of evaluation metrics, *Backward Transfer* and *Forward Transfer*, to evaluate models learning over a continuum of data. Backward transfer is calculated only once after all data is observed. The calculation of

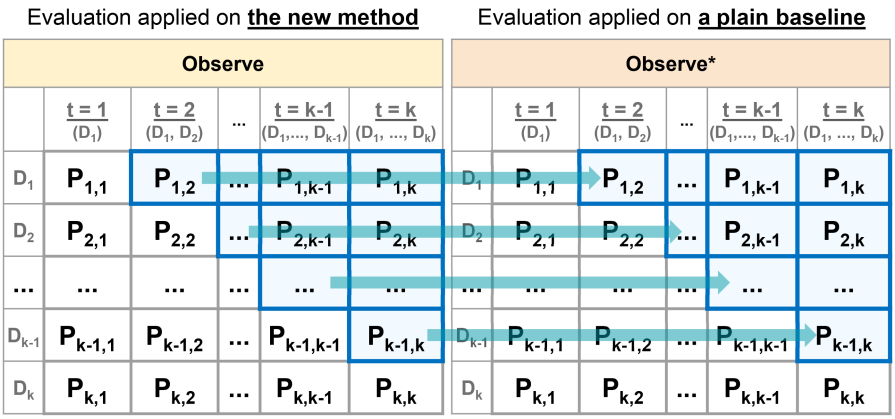

Fig. 3. An illustration of *Old Transfer* metric.

Forward transfer includes the performance at the time of model weights' random initialization. We argue that such two metrics have issues with not precisely estimating the capability of transferring knowledge. For one thing, the memorization of old data should be maintained throughout historical time, instead of only the oldest time or the latest time. For the other, Backward transfer and Forward transfer are not comparable with one another nor at the same scale because the performance scores at the time of random model initialization are in general low, which makes forward transfer exceptionally high when a model does perform forward transfer on new data.

For better generalization and robust evaluation of knowledge transfer in the context of various data drifting, we extend the metrics of backward and forward transfer in the following ways. The evaluation of the capability of knowledge transfer in data drifting requires the proposed *Old Transfer* and *New Transfer* to compare two learners. After a learner is fully trained on the i-th batch $data_i$ (abbrev. D_i), we measure the performance scores $P_{i,j}$ on test sets of every batch of data, i.e., $D_1, D_2, \ldots, D_k$, and obtain $P_{1,i}, P_{2,i}, \ldots, P_{k,i}$, where $P_{i,j}$ is the performance on data D_i after observing data D_j. Given a novel learner and a baseline learner, we refer to the performance of the former as $P_{i,j}$, and that of the latter as $P^*_{i,j}$. Assume there are k batches of data arriving at time, $t = 0, t = 1, \ldots,$ till $t = k$, we define old transfer and new transfer:

$$Old\ Transfer = avg\left(\sum_{i=1}^{k-1}\sum_{j=i+1}^{k-1}\left(P_{i,j} - P^*_{i,j}\right)\right),$$

$$New\ Transfer = avg\left(\sum_{i=2}^{k-1}\sum_{j=i-1}^{k-1}\left(P_{i,j} - P^*_{i,j}\right)\right).$$

We create Figures 3 and 4 to elaborate how *Old Transfer* and *New Transfer* are obtained, respectively. Old transfer is to quantify to what extent old data be forgotten by a model. Hence, for every pair of data batches D_i and D_j and $i < j$, we calculate the performance. In other words, $i < j$ means that a model is trained on new data D_j and is tested on old data D_i. On the contrary, new transfer estimates the degree of a model's adaption to new data. For each pair of data batches D_i and D_j and $i = j + 1$, we calculate $P_{i,j}$, indicating the performance that a model is trained on a batch D_j and tested on its immediate next batch D_{j+1}. By obtaining scores of old transfer and new transfer between a baseline learner and a novel learner, one would find it practical to justify the novel learner's effectiveness in transferring knowledge in data drifting. The larger the scores, the better a novel learner outperforms its baseline counterpart. Note that, for clearly presenting

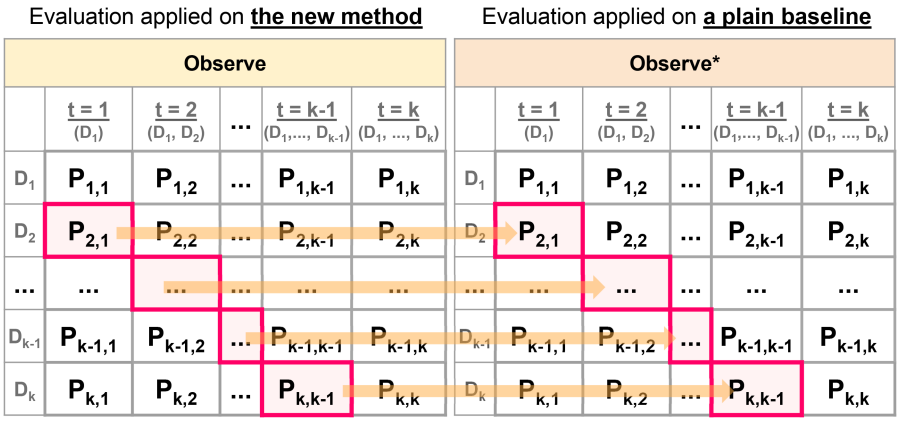

Fig. 4. An illustration of *New Transfer* metric.

Table 1. Differences between "Old Transfer" and "Backward Transfer"

Aspect	**Old Transfer (Proposed)**	**Backward Transfer (Existing)**
Equation	$\text{avg}(\sum_{i=1}^{k-1}\sum_{j=i+1}^{k-1}(P_{i,j}-P^*_{i,j}))$	$\text{avg}(\sum_{i=1}^{k-1}P_{k,i}-P_{i,i})$
Calculation Period	Throughout learning, comparing performance on previous batches before and after observing each subsequent batch	Only calculated once, after all data has been observed
Comparison Base	Compare performance of a novel learner to a baseline on old data following the observation of each subsequent batch	Compare performance on earlier batches before and after the entire training process
Learner Variability	Take into account the variability between a novel learner and a baseline learner, considering both their performances on older data after observing new data	Only consider the performance of a single model on old data after observing all data
Criticism	Backward Transfer neglects the gradual progression of how old data is remembered or forgotten throughout the learning journey by evaluating only after all data is observed, potentially concealing periods of forgetting and relearning during the learning process.	

the differences among these metrics, we create Tables 1 and 2, in which the equations of metrics are provided, to concretely compare the proposed old transfer and the existing backward transfer, and to compare the proposed new transfer and the existing forward transfer, respectively. In these two tables, the comparisons are on various aspects, including calculation period, comparison base, learner variability, and criticism.

Discussion. One concern about these two proposed metrics is that they require a test set for each time step (probably also a validation set for each time step), which may not be the case in real-world applications. First, regarding forward transfer, it appears there might be a slight misunderstanding. This metric does not necessitate storing a test set for every time step. Rather, it involves applying the model trained at time step t to the test set from time step $t+1$. This procedure does not entail retaining multiple test sets across all time steps and thereby is not hindered by the issues raised. However, for backward transfer, our initial methodology requires the availability of a test set for each time step, which could impose storage challenges. Yet, there are pragmatic ways to navigate this, such as storing a modestly-sized test set for each time step that is adequately representative, hence maintaining a balance between storage efficiency and experimental rigor. Alternatively, backward transfer could be modified to involve inference using the model from time

Table 2. Differences between "New Transfer" and "Forward Transfer", in which $\bar{b}$ is the Vector of Test Accuracies for Each Batch at Random Initialization

Aspect	**New Transfer (Proposed)**	**Forward Transfer (Existing)**
Equation	$\text{avg}(\sum_{i=2}^{k-1}\sum_{j=i-1}^{k-1}(P_{i,j} - P^*_{i,j}))$	$\text{avg}(\sum_{i=2}^{k} P_{i-1,i} - \bar{b}_i)$
Calculation Period	Throughout learning, comparing performance on new data after observing each previous batch	Include the model performance at the time of weights' random initialization, and calculated throughout training
Comparison Base	Compare performance of a novel learner to a baseline on new data, given the experiences from observing previous batches	Compare performance on subsequent batches to the model's performance at random initialization
Learner Variability	Consider the differential performance between a novel learner and a baseline learner on new data, accommodating an understanding of adaptive capabilities from observing previous data	Only contemplate the single model's adaptive capacities with reference to its initial, untrained state
Criticism	Forward Transfer is criticized for the inclusion of initial untrained performance, which is typically low and thus results in seemingly high adaptability scores. Furthermore, the inclusion introduces a non-comparability and scaling discrepancy with Backward Transfer, limiting their joint utility in holistically evaluating a model's adaptive learning performance.	

Table 3. Summary of the Four Generated Incremental Drift Datasets

Dataset name	drift type	# of batch	item # per batch	train:val:test	input format	task target
Aging	Covariate	5	2276	8:1:1	64-64 image	gender classification
Pose	Covariate	5	23708	8:1:1	64-64 image	gender classification
Amazon Review A	Actual	4	3175	8:1:1	tabular	semantic classification
Amazon Review C	Concept	4	3175	8:1:1	tabular	semantic classification

step t only on the test sets from the most recent m time steps, i.e., from $t-m$ to $t-1$. This approach would alleviate the necessity for extensive storage while still providing a relevant and insightful evaluation of the model's ability to generalize from its accumulated knowledge to previous tasks.

4 INCREMENTAL DRIFT DATA GENERATION

We generate synthetic datasets with incremental covariate drift, incremental actual drift, and incremental concept drift. In each generated dataset, instances are divided into multiple *batches* $B_0, \ldots, B_i, \ldots, B_k$, where $B_i = \{(x_j, y_j)\}_{j=1}^{n}$, k is the number of batches, and n is the number of instances in each batch B_i. A training model will observe, *batch by batch*, at different timestamps, i.e., $t_0, t_1, \ldots, t_i, \ldots, t_k$. The drifting behaviors in the synthesized data happen incrementally between every two *consecutive* batches over time. We summarize the four generated incremental drift datasets in Table 3.

4.1 Covariate Drift Data

In covariate data drifting, given two timestamps t_0 and t_1, we require $P(X)_{t_0} \neq P(X)_{t_1}$ while maintaining $P(y|X)_{t_0} = P(y|X)_{t_1}$. To create $P(X)$ drifts incrementally between batches of data, we provide two approaches. One is a *feature-based* approach, and the other is a ***generative adversarial network*** **(GAN)** based approach. In the meanwhile, by fixing the classification goal among all data batches, we can ensure $P(y|X)$ remains.

4.1.1 Feature-based Approach. We leverage an *ordinal* feature o in the original dataset. By grouping feature values of o into multiple disjoint sets in ascending order, data instances whose values of feature o belong to the same set are assigned to the same batch B_i. Hence, a model trained

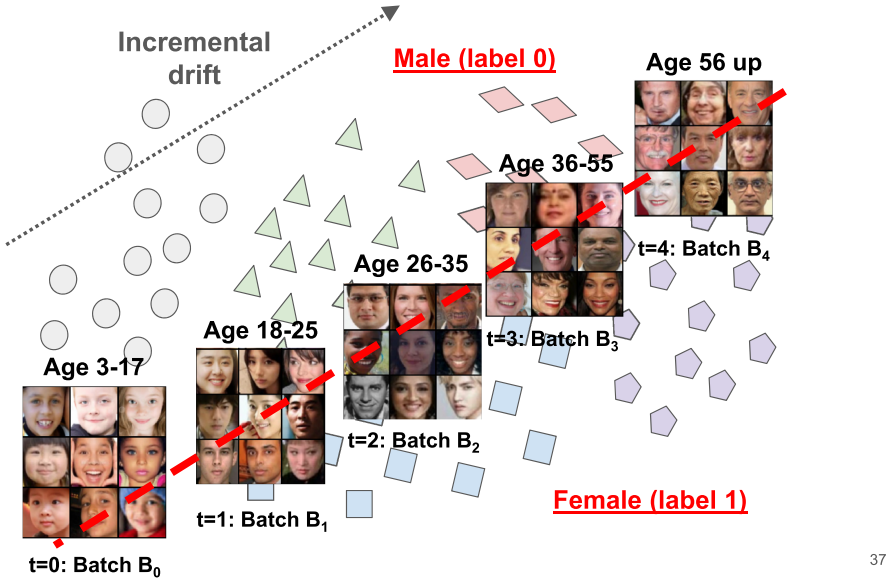

Fig. 5. An illustration of *Aging dataset* with incremental covariate drift. Different shapes (e.g., circles, triangles, rectangles) represent faces with different age intervals.

batch by batch will experience an incremental covariate drift with respect to feature o. Below we introduce the *Aging Dataset* based on a feature-based approach. Note that any existing dataset with an ordinal feature can use the same way to create a dataset with incremental covariate drift.

We utilize the UTKFace[2] [52] dataset to create the *Aging Dataset*. The UTKFace dataset contains roughly 20,000 human face images ranging from 0 to 116 years old. We use the age feature, as an ordinal feature, to divide the face images into five batches, i.e., $B_0, B_1, \ldots, B_4$, each of which contains images aged between 3-17, 18-25, 26-35, 36-55, and 56 up, respectively. On the other hand, the target task is to do gender classification. This guarantees $P(y|X)$ to be consistent among all batches. Figure 5 is an illustration of how age spans change among batches over different timestamps, in which the gender distribution is maintained, to construct the incremental covariate drift data.

4.1.2 GAN-based Approach. Even though *Feature-based* obtains success in generating incremental $P(X)$ drifts, we cannot always expect every dataset contains proper ordinal or continuous features, like age, to create covariate drift data. Therefore, we propose to use the *semantic interpolation* capability of generative adversarial networks (GAN) [18] to match such incremental changes from scratch. GAN has demonstrated its great effectiveness in generating realistic images [4, 21, 22]. Not only does the model generate high-quality images but its latent space also shows interpretability. Recent studies [37, 38] have worked on latent semantic interpretation. They aim at finding meaningful directions in the latent space in either a supervised or an unsupervised manner. By moving the latent code in a certain direction, one is able to take control of the traits of output images, effects of which include a gradual change in lighting condition in scene synthesis, or the extent of a smile on faces.

We use SeFa[3] [38], the state-of-the-art unsupervised method of GAN-based semantic interpolation, to create the *Pose Dataset* with incremental covariate drift. We first apply a StyleGAN [22]

[2]https://susanqq.github.io/UTKFace/
[3]https://github.com/genforce/sefa

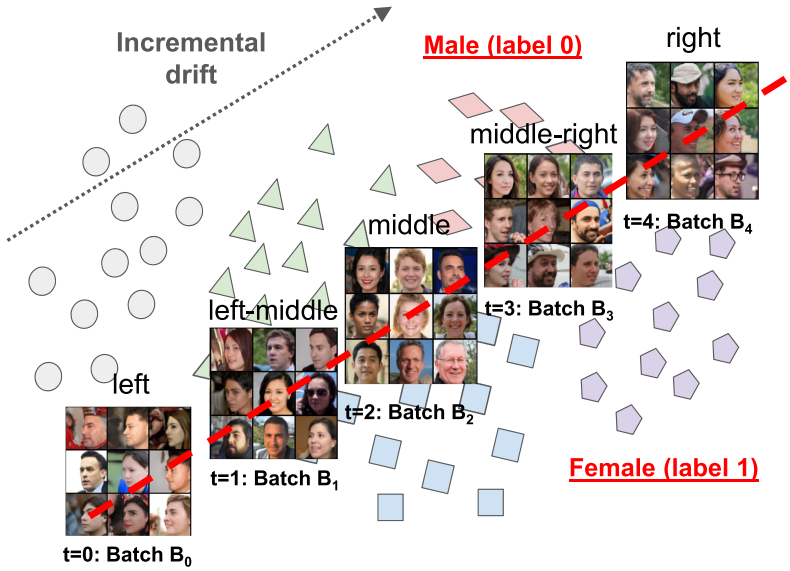

Fig. 6. An illustration of *Pose dataset* with incremental covariate drift. Different shapes (e.g., circles, triangles, rectangles) represent faces with different turnings.

model trained on the **Flickr-Faces-HQ Dataset (FFHQ)**[4] to the SeFa dataset. By factoring the 0-1 layer of the generator, we segment it into 20 fragments. Finally, we retrieve the 1, 6, 9, 13, 19 steps of images as the five batches of images, i.e., $B_0, B_1, \ldots, B_4$, in the Pose dataset. Each batch contains 8,278 images. Likewise, $P(y|X)$ remains consistent as the goal among all data batches is gender classification. Figure 6 shows an illustration of the compiled Pose dataset. Incremental changes in face turnings batch by batch can be observed while the gender distribution is kept.

Note that, aside from image data, GAN can be used to generate tabular data [50]. Therefore, we highlight the potential of generalizing GAN's semantic interpolation to generate incremental $P(X)$ drifts in future work. By revealing the underlying semantic vector of a pre-trained GAN, one can create incremental $P(X)$ drifts.

4.2 Actual Drift Data

To create $P(y|X)$ drifts incrementally between consecutive data batches, we adopt the *feature-based* approach. Given a dataset with an ordinal label o, a threshold τ can be used to convert the task to be a binary classification, where label= 0 if $o \leq \tau$ while label= 1 if $o > \tau$. By changing the threshold τ from small to large between consecutive batches, incremental $P(y|X)$ drifts can be created. In the meanwhile, $P(X)$ remains fixed by randomly dispensing data instances into batches. Note that any existing dataset with an ordinal label can be used in the same way to create a new dataset with incremental actual drift.

We create the Amazon Review Actual Drift dataset (*Amazon Review A*) from the Amazon Review dataset[5] [32]. To let the data better fit our goal, we retrieve product reviews from only four categories: Art Craft, Digital Music, Lawn & Garden, and Software. Each data instance contains a text review and a score ranging from 1 to 5. We draw an equal number of 2,540 reviews for each

[4]https://github.com/NVlabs/ffhq-dataset
[5]https://nijianmo.github.io/amazon/

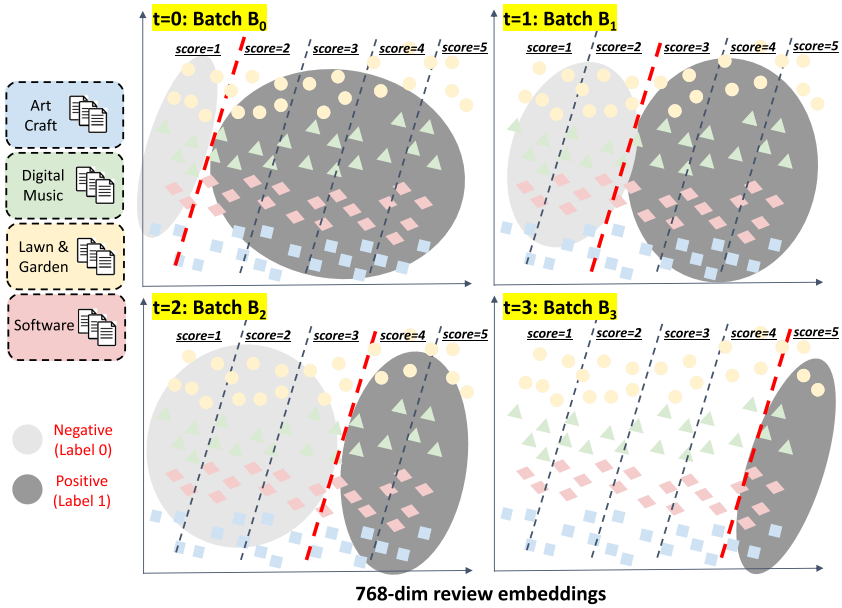

Fig. 7. An illustration on *Amazon Review A* dataset with incremental actual drift.

category. We encode each text review into a 768-dimensional embedding by a pre-trained BERT[6] [9]. As for the 1–5 scores, we divide them into two classes: `like` and `dislike` based on a threshold τ. Scores lower than the threshold go to the `dislike` class whereas scores above the threshold go to the `like` class. By adjusting the thresholds from 2 to 4, we can generate four data batches B_0, B_1, B_2, B_3 that map the incremental actual drifts in $P(y|X)$. In the meanwhile, $P(X)$ remains the same since all reviews are equally sampled from the four product categories. Figure 7 shows an illustration of how Actual Drift happens on review embeddings among data batches at different timestamps.

Note that we strategically define the threshold, τ, with reviews scoring below it designated as "dislike" and those above as "like", thereby converting the problem into a binary classification task. The progression of τ over various time points aims to simulate actual drift by gradually escalating this threshold. It is crucial to elucidate that this design choice is primarily a hypothesis devised to generate an incremental actual drift dataset, while also offering a framework of design that can be referenced in similar experimental contexts. As long as the probability distribution $P(y|X)$ shifts, alternative hypotheses, such as assigning different thresholds for different products, could be equivalently formulated and tested. The concrete choice of threshold and labeling strategy was shaped with the intention of providing a clear, comprehensible, and reproducible methodology for simulating actual drift in a widely recognized dataset, thereby facilitating a robust evaluation of the proposed methods under consistent and transparent conditions.

4.3 Concept Drift Data

We compile the incremental concept drift dataset based on the feature-based approach. The objective is to simultaneously create the drifts of $P(X)$ and $P(y|X)$ incrementally between consecutive data batches. Given a dataset with a categorical feature c and an ordinal label o, a threshold τ can

[6] https://huggingface.co/bert-base-uncased

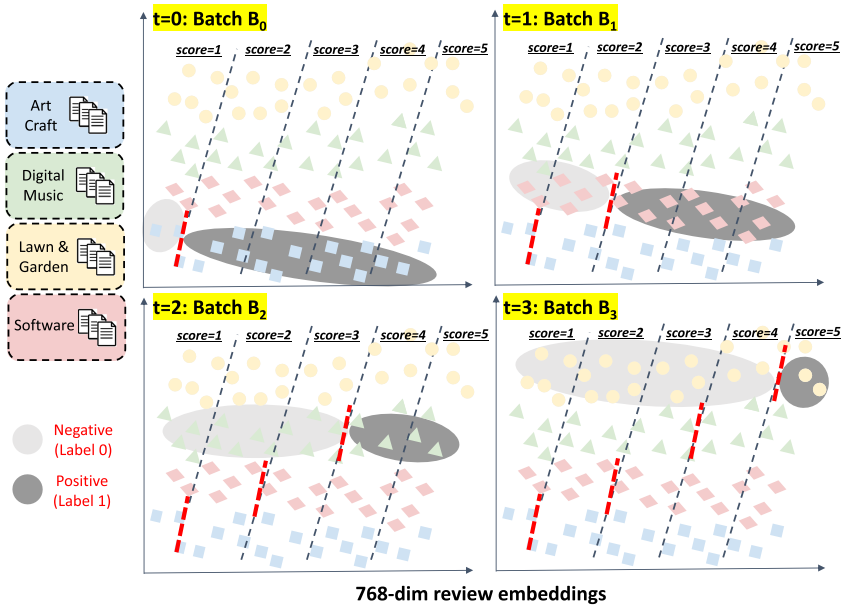

Fig. 8. An illustration on *Amazon Review C* dataset with incremental concept drift.

be created to convert the task to binary classification, where label= 0 if $o \leq \tau$ while label= 1 if $o > \tau$. By changing the threshold τ from small to large between consecutive batches, data instances with incremental $P(y|X)$ drifts can be generated. In the meanwhile, since data instances with various values of a certain categorical feature tend to exhibit different distributions, $P(X)$ drifts on instances can be produced through the categorical feature c. We confine each data batch to contain data instances belonging to a specific value of categorical feature c. Note that any existing dataset with a categorical feature and an ordinal label can be used in the same way to create a new dataset with incremental concept drift.

Similarly, we create the Amazon Review Concept Drift dataset (*Amazon Review C*) from the Amazon Review Dataset [32]. In addition to adjusting the threshold τ between data batches to create $P(y|X)$ drifts, we create $P(X)$ drifts by changing the review categories, i.e., Art Craft, Digital Music, Lawn and Garden, and Software in different batches. Data instances with different review categories are assigned to different batches. Eventually, we can produce four batches B_0, B_1, B_2, B_3 for the drift of $P(X)$, and each of which has its own threshold τ that determines the drift of $P(y|X)$. Figure 8 provides an illustration of how incremental concept drift happens on review embeddings among data batches at different timestamps.

4.4 Evaluation on Incremental Drift

We aim at examining whether or not the four generated datasets do contain incremental drifts. We expect that if the data contains incremental drifts, a learner's prediction performance will be diminished when shifting from one batch to another. To construct the evaluation, we finetune a base neural network model that is trained on each training batch over a fixed number of epochs (50 for the first dataset, and 30 for the remaining three datasets). When each training epoch is done, the learner is tested on the test set of that batch. Note that the detailed model configuration and experimental settings are presented in Section 6.1. The performance scores in terms of classification accuracy over all epochs in the four generated datasets are reported in Figure 9. We can see

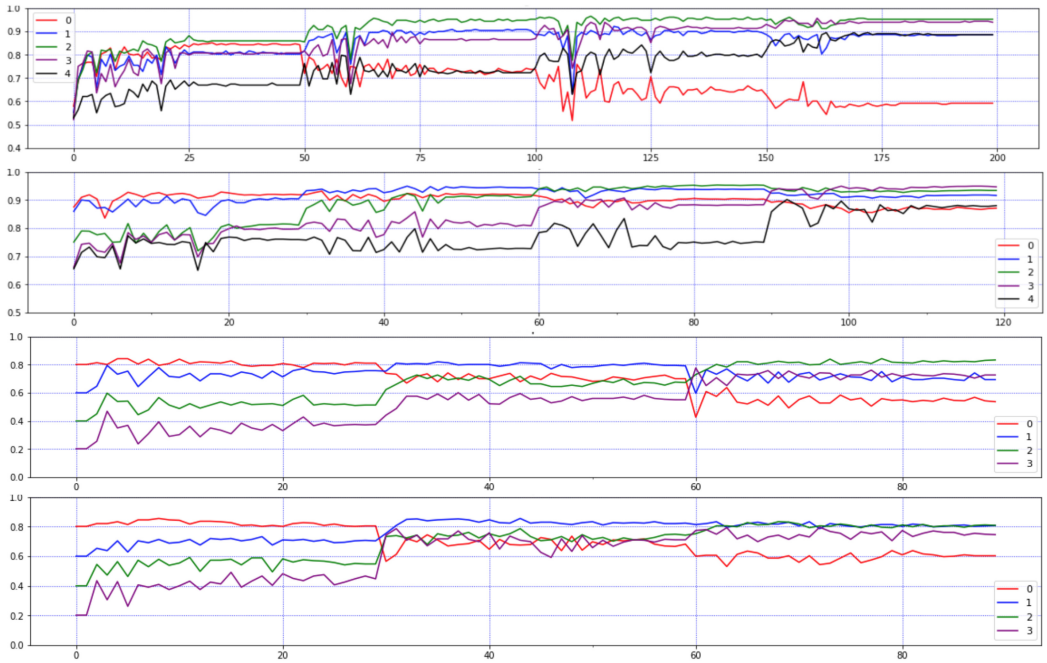

Fig. 9. Finetuning accuracy (y-axis) vs. the training epoch number (x-axis) on the four datasets (from top to down): Aging dataset (50), Pose dataset (30), Amazon Review A (30), and Amazon Review C (30), where the number in the bracket is the number of epochs per batch. In the legend, the number from 0 to 3 or 4 refers to batches $B_0, B_1, \ldots, B_3$ or B_4. For example, the red curve shows the accuracy values reported from training on the first batch B_0 over epochs.

that once the learner moves from one batch to the next one, in every dataset, the accuracy values show significant drops. It is because the learner moves to train on the next batch with a drifted concept, causing the gradual forgetting of the previous batch's concept. Such results indicate that incremental drifts exist among batches in the four generated datasets.

Note that our intention behind employing GANs to synthesize datasets was to create a controlled experimental environment where we could meticulously modulate the drift characteristics, thereby facilitating a nuanced exploration of the algorithms under distinct incremental drift scenarios. It is also pertinent to note that while our proposed datasets demonstrate evident shifts and complexities, the perennial challenge remains that no synthetic dataset can wholly encapsulate the multifaceted nature of real-world drifts. Consequently, while we argue that our datasets, generated with considered applications of GANs, present a significant step forward in approximating real-world complexities, we concede and underscore that they are not an exhaustive representation of all possible real-world scenarios. Nonetheless, we posit that they serve as a valuable tool in bridging the gap between conventional synthetic datasets and the unpredictable intricacies observed in real-world applications.

5 MODEL COMPARISON

Given a dataset with incremental data drifts, we aim to investigate how recent advances in knowledge transfer between domains/tasks can be adopted for the predictions. We find that *Domain Adaptation* [47] and *Lifelong Learning* [8] are the two most relevant approaches to model incremental data drifting. Both approaches can deal with learning knowledge from a task/domain and hav-

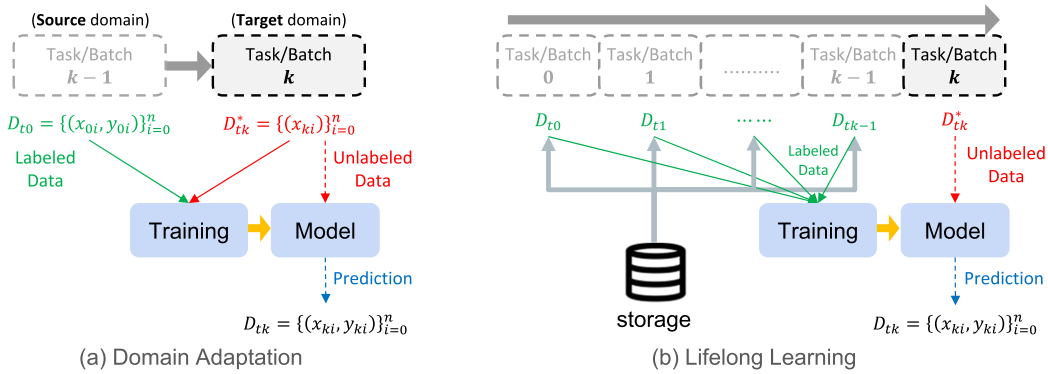

Fig. 10. Elaboration of domain adaptation and lifelong learning for incremental data drifting. Domain-adaptation methods train on both labeled data in batch $k-1$ and unlabeled data in batch k, and predict on unlabeled data in batch k. Lifelong learning trains on all previously labeled data in batches $0, 1, \ldots, k-1$ and predict on unlabeled data in batch k.

ing it transferred to another task/domain, and we can treat the predictions on the drifted batches as sequential tasks with domain shifts. We also study how well can the methods on *Concept Drift Adaption* [15] be utilized to make predictions on incremental data drifting.

5.1 Domain Adaptation

Domain adaptation deals with the prediction task, in which datasets are collected from two domains with different distributions, i.e., the source domain and the target domain. It aims at transferring knowledge learned from the source domain to the target domain through adversarial training. As shown in Figure 10(a), a learner is trained and adapted to an unlabeled target domain leveraging both labeled data in the source domain and unlabeled data in the target domain. Since domain adaptation is capable of handling feature discrepancies, it provides some potential to deal with new concepts with incremental covariate drifts. By treating source and target domains as consecutive batches B_{k-1} and B_k, we are allowed to seamlessly exploit domain adaptation methods to tackle incremental data drifting. Below we compare three typical methods of domain adaptation.

- **DANN**: **Domain-Adversarial Neural Network (DANN)**[7] [16] achieves domain adaptation by generating features that cannot be told from source to target domain. In addition to minimizing the label prediction loss for source-domain data, DANN minimizes the domain classification loss for all instances and produces domain-invariant features by a gradient reversal layer, which ensures that the feature distributions over two domains are made similar.
- **MCD**: **Maximum Classifier Discrepancy (MCD)**[8] [36] is an unsupervised domain adaptation algorithm that considers task-specific decision boundaries between classes. The adversarial training model consists of a feature generator and two label classifiers. By maximizing the discrepancy between two classifiers on target-domain samples, and generating latent features that minimize the discrepancy, MCD aligns source- and target-domain data distributions.
- **GST**: **Gradual Self-Training (GST)**[9] [27] focuses on data that the domain shift happens gradually. The goal is to adopt an initial classifier trained on the labeled source domain

[7] https://github.com/fungtion/DANN
[8] https://github.com/mil-tokyo/MCD_DA
[9] https://github.com/p-lambda/gradual_domain_adaptation

given unlabeled intermediate domains that shift gradually in distribution towards an unlabeled target domain. GST utilizes self-training to model gradual shift. The classifier first generates pseudo-labels for the successive domain. High-confident pseudo-labeled samples are considered to train a regularized supervised classifier. By iterating this process when intermediate-domain data arrives, GST can gradually adapt to the final target domain. Here we apply GST every time a new data batch arrives.

— **ARM**: **Adaptive Risk Minimization (ARM)**[10] [51] offers a novel approach in the domain generalization problem setting, where training data is categorized into domains, and potential test-time shifts into new domains or domain distributions are anticipated. ARM creates models that can adapt during test time to domain shifts, using unlabeled test points. ARM aims to optimize the model to proficiently utilize the unlabeled adaptation phase to manage domain shifts by meta-learning an adaptable model from a set of training domains that correspond to training batches in our setting. Similarly, we apply ARM every time a new data batch arrives.

5.2 Lifelong Learning

Lifelong learning is performed on a sequence of tasks whose data distributions changed are presented chronologically. As illustrated in Figure 10(b), a lifelong learner needs to do accurate predictions on the new task (on data batch B_k) while learning and maintaining the performance on historical tasks (on $B_0, \ldots, B_{k-1}$), given that only a limited amount of previously seen data batches and models can be saved. An extreme decrease in performance on old tasks is called *catastrophic forgetting*. Since lifelong learning can effectively mitigate catastrophic forgetting, it is expected to memorize all historical knowledge with covariate drifts. By regarding tasks with historically labeled data as sequential training batches, and the next task as the target batch B_k being predicted, lifelong learning can be a proper approach to deal with incremental data drifting. We experimentally compare two typical lifelong learning methods.

— **EWC**: **Elastic Weight Consolidation (EWC)**[11] [25] prevents catastrophic forgetting in the setting of lifelong learning by flexibly decreasing the learning on certain weights according to how they positively contribute to historical tasks. EWC devises a novel regularization term that can reflect the importance of every single model parameter learned from historical tasks, and penalize the weight updates that attempt to modify important parameters. Putting EWC to the learning with incremental drift data is expected to effectively maintain the knowledge learned from observed batches.

— **GEM**: **Gradient Episodic Memory (GEM)**[12] [29] mitigates catastrophic forgetting by maintaining an episodic memory that stores a subset of the observed instances from historical data. By computing the inner product between the loss gradient vector of the data in the memory and the current update derived from new data, GEM can diagnose whether the loss at historical tasks is increased. If it is, GEM finds an alternative gradient whose parameter update is unlikely to hurt the performance on past tasks, leading to maintaining past learned knowledge.

5.3 Concept Drift Adaptation

The adaptive approach to handling concept drifts is a kind of incremental learning that is able to adapt to the evolution of the data generation process over time. The predictive models update

[10] https://github.com/henrikmarklund/arm
[11] https://github.com/ariseff/overcoming-catastrophic
[12] https://github.com/facebookresearch/GradientEpisodicMemory

online during their operations to react to concept drifts. When to start the adaptive process and how to deal with the changes are two essential issues. For the first issue, one can blindly and regularly trigger the adaptive process without knowing when the concept drift happens [42], e.g., re-training the model every day. Another method is to start the adaptive process when the concept drift is detected by the drift detection system [41]. As for the second issue, a straightforward way is to retrain a new model with the newest data. The model ensemble is also a popular method to mitigate performance decay caused by concept drift. A new model is trained and added to the ensemble, and the old model with the worst performance on the newest data is removed [10]. Given that our data setting is batch-wise and requires no drift detection, we consider the following typical concept drift adaptation method.

— **Learn^{++}.NSE**: **Learn^{++}.NSE (LNSE)**[13] [10] is a model ensemble method, in which classifiers are trained on different data batches. A new classifier is trained on the latest data batch if the error rate of the previous classifier in the ensemble exceeds 0.5. In addition, dynamically weighted majority voting is applied based on the performance of the latest data of each classifier.

6 EXPERIMENTAL EVALUATION

We answer six questions. (1) Which approaches among domain adaptation, lifelong learning, and concept drift adaptation perform better? (2) Can domain adaptation methods perform well when new data arrives? (3) Will domain adaptation methods decrease their performance on Old data when they try to align domain drifts? (4) Can lifelong learning methods perform well on historical data? (5) Will lifelong learning methods perform better given that they can alleviate catastrophic forgetting? (6) How do different approaches perform on real concept drift datasets?

6.1 Experimental Settings

Baselines. We consider three baselines that do not handle drifting.

— **Finetune**: The model is sequentially trained on the immediately previous data batch and fine-tuned on the current batch without any advanced knowledge-transfer techniques applied. Methods belonging to either domain adaptation or lifelong learning can have such finetuning versions.
— **Joint**: The model is trained using a *part* of historical data instances stored before the prediction batch. For a fair comparison, we use the same amount of data as the memory size in GEM [29] to create the Joint model.
— **Joint-full**: The model is trained on *all* of historical data instances. All instances in past batches are used simultaneously to train the model. Therefore, Joint-full can be seen as an *upper bound* for Lifelong Learning methods.

Note that one may question how batch-wise data is significantly different from a data stream. A naive solution through updating the model with the mean of the gradient of the batch should work for batch-wise data. In fact, such a naive solution is analogized with the "Finetune" baseline that we included for comparison and discussion. This Finetune approach was adopted to ensure a balanced and rigorous analysis, accounting for both conventional and alternative methodologies in managing batch-wise data amidst concept drift.

Datasets and Splittings. We use the four generated datasets, Aging, Pose, Amazon Review A, and Amazon Review C, as presented in Table 3. To understand how well different approaches can be utilized to real-world concept drifts, we further run the experiments on two real datasets, Gas

[13] https://github.com/gditzler/IncrementalLearning

Table 4. Summary of the Two Real Datasets

	# data	# feature	# Class	# batch	# data per batch	train:val:test
Gas Sensor Array	13,910	128	6	10	variant by time	8:1:1
Forest Covertype	581,012	54	7	7	83000	8:1:1

Sensor Array Drift dataset[14] [12] and Forest Covertype dataset[15] [3], whose statistics is provided in Table 4. In the generation of each synthetic dataset and in the experiments, each data batch contains not only a validation set, which is used for model selection, but also a test set to display final performance. The idea behind such kind of data splitting is to divide the data in each distribution into validation and test, assuming the access to a small validation set that shares the same distribution with the test. This approach allows for robust model training, hyperparameter tuning, and eventual evaluation of the model's generalization performance of incremental data drifting.

Base Learner Settings. For a fair comparison, the same neural network architecture is used as the base learner for all compared methods running under the same dataset. If the input data is the image, a typical 4-layer convolutional neural network channeled 3-32-64-128 is used. As for tabular data, we employ a multi-layer perceptron with 4 hidden layers of 64 neurons each. We add dropout with 0.2 probability to each layer. The activation function ReLU is used in all hidden layers. We use Adam [24] to optimize all models. All methods use the standard cross-entropy loss for the classification tasks.

Model Selection. The ways to tune hyperparameters of all comparing methods in **domain adaptation (DA)**, **lifelong learning (LL)**, and **concept drift adaptation (CDA)** follow the respective studies unless further specification. We perform model selection with hyperparameter tuning using the validation set within each data batch. In DANN, The domain adaptation parameter λ (the trade-off between the prediction loss and the negative of the domain loss) is defined as follows: $\lambda_p = \frac{2}{1+e^{-10p}} - 1$, where p is the training step linearly changing from 0 to 1. In MCD, the number of times to repeat the process of minimizing the discrepancy loss is tuned within $\{1, 3, 5, 7\}$. In EWC, the scaling factor, which reflects the importance of the old task, to have better performance, is tuned within $\{800, 1200, 1800, 2400, \ldots, 4800\}$. For GEM and Joint, a subset of instances in each historical batch is randomly sampled and stored with a ratio α. We tune $\alpha = \{0.025, 0.05, 0.075, 0.1\}$. Note that the number of instances per batch varies in Gas Sensor Array Drift dataset because batches are determined according to fixed time periods. We define n_s as the size of the smallest data batch, and the actual number of instances to be stored for each batch is $n_s \times \alpha$.

Evaluation Metrics. We utilize the metrics described in Section 3, including Old Transfer and New Transfer. Since both require baselines to have relative performance scores, here we consider *Finetune* as the baseline learner. Note that this is why the scores of such two criteria in "finetune" rows in the resultant tables show 0 across all datasets. We report the average results on the test sets over ten different seeds, along with the standard deviation.

6.2 Results and Discussion

Covariate Drift. The results for incremental covariate drift on the generated Aging and Post datasets are shown in Tables 5 and 6. We find out that when covariate drift happens, lifelong learning-based algorithms succeed in preserving the knowledge in old data. Therefore, they are proven to be effective when handling situations where old concepts reoccur in the future.

[14]https://archive.ics.uci.edu/ml/datasets/gas+sensor+array+drift+dataset
[15]https://archive.ics.uci.edu/ml/datasets/covertype

Table 5. Results on the Aging Dataset

		Old	New
Baseline	finetune	0.0000 ± 0.00	0.0000 ± 0.00
	joint	0.0310 ± 0.03	0.0018 ± 0.01
	joint-full	0.0908 ± 0.08	−0.0227 ± 0.02
LL	GEM	**0.0370** ± 0.03	−0.0078 ± 0.02
	EWC	**0.0231** ± 0.04	−0.0168 ± 0.03
DA	MCD	−0.0147 ± 0.03	**0.0220** ± 0.01
	DANN	0.0025 ± 0.03	**0.0038** ± 0.02
	GST	−0.0185 ± 0.01	**0.0033** ± 0.02
	ARM	0.0032 ± 0.02	**0.0079** ± 0.02
CDA	LNSE	−0.1763 ± 0.08	−0.2309 ± 0.04

Table 6. Results on the Pose Dataset

		Old	New
Baseline	finetune	0.0000 ± 0.000	0.0000 ± 0.000
	joint	0.0099 ± 0.013	0.0043 ± 0.009
	joint-full	0.0212 ± 0.017	−0.0057 ± 0.013
LL	GEM	**0.0126** ± 0.009	0.0058 ± 0.010
	EWC	**0.0107** ± 0.011	−0.0061 ± 0.008
DA	MCD	−0.0377 ± 0.026	**0.0199** ± 0.009
	DANN	−0.0175 ± 0.015	**0.0208** ± 0.010
	GST	−0.0063 ± 0.010	**0.0012** ± 0.009
	ARM	0.0002 ± 0.012	**0.0314** ± 0.008
CD	LNSE	−0.3054 ± 0.044	−0.2271 ± 0.051

Furthermore, GEM consistently outperforms joint training in Old Transfer, given that they both store the same amount of old data. However, both GEM and EWC have very limited improvements in covariate drift in New Transfer. EWC aggravates more when intransigence happens. The reason for this difference is because of the innate shift of feature space in covariate drifting data; as data keep exploring new spaces without overlaps, the burdens of mitigating data discrepancies by models increase. Such a weak adaptation to unseen covariate drift data is especially obvious for lifelong learning models because they cannot utilize any unlabeled data in the batch being predicted.

On the other hand, all domain adaptation methods perform well on New Transfer measured for incremental covariate drift, as exhibited in Tables 5 and 6. They succeed in aligning unlabeled data shifting in the feature domain. To be more specific, MCD is slightly better than DANN and ARM since it not only aligns the feature spaces but also considers the potential differences in decision boundaries between domains/batches. Although GST and ARM also perform better than lifelong learning-based methods on New Transfer mostly, it has limited effect compared with other domain adaptation methods because the degree of gradual drift is not large enough in the generated datasets. In other words, GST and ARM only perform well on data that drift very slowly. We also find out that all domain adaptation-based methods suffer from performance degradation in Old Transfer. A possible reason for this is that when feature spaces are continually adapted, models are guided away from the old concepts while in meanwhile models pay much attention to newer concepts and unlabeled data in the batch being predicted.

Actual Drift. We report the results on incremental actual drift using the generated *Amazon Review A* dataset in Table 7. We can find that on lifelong learning methods, GEM performs well while

Table 7. Results on the Amazon Actual Drift Dataset

		Old	New
Baseline	finetune	0.0000 ± 0.000	0.0000 ± 0.000
	joint	0.0264 ± 0.014	−0.0028 ± 0.011
	joint-full	0.0950 ± 0.042	−0.0618 ± 0.058
LL	GEM	**0.0283** ± 0.020	−0.0049 ± 0.013
	EWC	0.0069 ± 0.014	0.0047 ± 0.008
DA	MCD	0.0180 ± 0.009	−0.0005 ± 0.011
	DANN	0.0013 ± 0.018	0.0025 ± 0.014
	GST	−0.0202 ± 0.020	**0.0133** ± 0.020
	ARM	0.0057 ± 0.017	**0.0097** ± 0.014
CDA	LNSE	−0.0216 ± 0.094	−0.2005 ± 0.088

EWC shows an obvious decline. In the comparison between GEM and joint training, GEM still obtains better performance given that they both use the same amount of storage to store old data. This helps us come to the insight that GEM is better at preserving old concepts when incremental actual drift happens. On the other hand, all domain adaptation methods fail in tackling incremental actual drift except gradual self-training. Consistent with the innate design of these methods, they showed no improvement when drifts only happened in decision boundaries rather than the feature spaces. This highlights the limitations of the domain adaptation methods: although all types of data drifting cause performance decline, they come into effect only when the drifts happen on $P(X)$. Interestingly, GST and ARM obtain the best performance in tackling incremental actual drift. We think data lying in the drift region may be filtered out because of the low prediction confidence. Therefore, it may be feasible to obtain a better decision boundary after retraining on the remaining pseudo-labeled samples, bringing better performance in predicting the next concepts.

Concept Drift. Table 8 exhibits the results of evaluating incremental concept drift based on the generated Amazon Concept Drift dataset. We can find that lifelong learning-based methods, in particular GEM, can successfully preserve old concepts. However, it also causes serious performance degradation in New Transfer. We speculate that this is because of the contradiction in decision boundaries between data batches. Lifelong learning approaches are hard to foresee and adapt to the drifted decision boundaries in the testing batch. Also, the better a lifelong learning-based method is in Old Transfer, the worse it is in New Transfer. On the other hand, domain adaptation-based methods, again, fail in New Transfer. Moreover, MCD and ARM decline even more in incremental concept drift, compared to incremental actual drift. We consider this happening due to the false decision boundaries MCD and ARM learned while trying to align feature spaces. Gradual self-training also fails because the drift of the data feature space is not gradual enough.

Real Data Drift. We present the results on real-world incremental data drifting in Tables 9 and 10. We find that lifelong learning-based methods lead to promising performance on both old and new transfers. In the case of Gas Sensor data, GEM nearly reaches the performance's upper bound (i.e., joint-full). On the other hand, while MCD has some success in Forest Covertype data, the improvements that lifelong learning methods make are more significant. This is because that new data come in a mixture of old data. Therefore, having old data memorized helps a learner to perform well when it recurs. This also highlights the importance of generating incremental datasets with different specific drift types. Only the old transfer and new transfer evaluated on incremental changed data can reveal a learner's true abilities to prevent forgetting and being able to adapt forward at the same time.

Table 8. Results on the Amazon Concept Drift Dataset

		Old	New
Baseline	finetune	0.0000 ± 0.000	0.0000 ± 0.000
	joint	0.0611 ± 0.034	−0.0498 ± 0.054
	joint-full	0.0864 ± 0.063	−0.0660 ± 0.057
LL	GEM	**0.0630** ± 0.045	−0.0579 ± 0.048
	EWC	−0.0011 ± 0.012	−0.0069 ± 0.014
DA	MCD	0.0456 ± 0.077	−0.0860 ± 0.122
	DANN	0.0318 ± 0.040	−0.0071 ± 0.042
	GST	−0.0609 ± 0.024	−0.0193 ± 0.026
	ARM	0.0494 ± 0.028	−0.0037 ± 0.025
CDA	LNSE	−0.1271 ± 0.159	−0.2184 ± 0.059

Table 9. Results on Gas Sensor Array Drift Dataset

		Old	New
Baseline	finetune	0.0000 ± 0.000	0.0000 ± 0.000
	joint	0.1413 ± 0.113	0.0428 ± 0.091
	joint-full	0.1882 ± 0.130	0.0478 ± 0.100
LL	GEM	**0.1675** ± 0.097	**0.0362** ± 0.085
	EWC	**0.0580** ± 0.077	**0.0044** ± 0.061
DA	MCD	−0.0241 ± 0.088	0.0047 ± 0.084
	DANN	−0.0135 ± 0.060	−0.0123 ± 0.120
	GST	−0.0101 ± 0.067	−0.0438 ± 0.079
	ARM	−0.0076 ± 0.057	−0.0070 ± 0.076
CDA	LNSE	−0.4270 ± 0.181	−0.4290 ± 0.143

Table 10. Results on Forest Covertype Dataset

		Old	New
Baseline	finetune	0.0000 ± 0.000	0.0000 ± 0.000
	joint	0.1586 ± 0.098	−0.0019 ± 0.024
	joint-full	0.3855 ± 0.090	0.1008 ± 0.055
LL	GEM	**0.1785** ± 0.086	−0.0042 ± 0.023
	EWC	**0.1131** ± 0.073	**0.0358** ± 0.020
DA	MCD	−0.1098 ± 0.091	0.0117 ± 0.023
	DANN	−0.1280 ± 0.100	−0.0652 ± 0.033
	GST	−0.0363 ± 0.098	0.0299 ± 0.027
	ARM	−0.0193 ± 0.093	0.0160 ± 0.022
CDA	LNSE	−0.1114 ± 0.146	−0.0664 ± 0.045

7 CONCLUSIONS AND DISCUSSION

In this work, we highlight and tackle the problem of incremental data drifting under covariate, actual, and concept types. While existing studies target instance-wise data streams, cannot properly evaluate models on knowledge transfer, do not work on specific drifting datasets, and have not investigated advanced learning approaches, we provide the first holistic attempt for learning with incremental drifting in the batch-wise data setting. We propose two novel metrics, old and

new transfer, to properly and robustly reflect the goodness of knowledge transfer between data batches. We also propose feature-based and GAN-based mechanisms to generate synthetic incremental drifting tabular and image datasets with explicit drift types. By properly aligning the tasks between the techniques for domain adaptation and lifelong learning in the scope of incremental data drifting, we experimentally compare their performance. We obtain insights that depict the underlying relationships among drifting types, knowledge preservation, and learning approaches. First, the lifelong learning approach, especially GEM, is good at preserving old knowledge in all kinds of datasets and across drift types, but fails in adapting to unseen new data. Second, the domain adaptation approach works well on adapting concept shift to the next data batch in incremental covariate drift, but hurts the performance in all other drift types as it aligns only feature spaces based on unlabeled data. Third, adapting to unseen data with incremental concept drift is the most challenging because both feature and label spaces are shifted, and thus none of domain adaptation and lifelong learning methods can work well. Fourth, when facing concept drift in real data, the lifelong learning approach is a better choice for both preserving old knowledge and adapting to new knowledge.

In this work, we only evaluate the performance of domain adaptation and lifelong learning methods in the presence of incremental data drifting. There is the potential to devise a novel method to tackle the issue of incremental data drifting. Indeed, the conundrum of incremental data drifting poses an interesting challenge in dynamic learning environments. Incremental data drift typically exhibits a certain regularity and exploring this regularity can potentially forge a path towards proactively predicting future data concepts, thereby offering a viable strategy to navigate through the issues posed by incremental data drifting. Taking actual drift as an example, this regularity can be discerned by understanding the variations in the class distribution of each data point across previous time instances. This suggests that for a data batch arriving at time t, we might train a model using the class distributions from the past m time points ($t - m$ to $t - 1$) as input, and the class distribution at the current time point t as the label. By doing so, during the inference stage, we could shift the input window by one time unit to consider class distributions from $t - m + 1$ to t, enabling the model to predict the instance's class distribution at time $t + 1$ and, potentially, estimate the concept at $t + 1$.

Obtaining the class distribution of a particular instance at various time points might be achieved by performing inference using classifiers trained at each respective time point. This creates a loop of continuous adaptation and learning, wherein the model not only learns from the drifting data but also predicts subsequent drifts, thereby preparing itself to adjust to future shifts. This systematic method ensures that the model is not only reactive but also proactive in its approach towards handling incremental data drifting, potentially reducing the lag between the occurrence of drift and the model's adaptation to it, and thus maintaining a robust predictive performance despite the dynamic data landscape. It's worth noting that this proposed method would necessitate thorough empirical validation to ascertain its effectiveness and applicability across diverse data drift scenarios. And certainly, this exploration can further enrich the discourse in the realm of handling incremental data drift.

REFERENCES

[1] Martin Arjovsky, Léon Bottou, Ishaan Gulrajani, and David Lopez-Paz. 2019. Invariant risk minimization. *arXiv preprint arXiv:1907.02893* (2019).

[2] Manuel Baena-García, José Campo-Ávila, Raúl Fidalgo-Merino, Albert Bifet, Ricard Gavald, and Rafael Morales-Bueno. 2006. Early drift detection method. (01 2006).

[3] J. Blackard and D. Dean. 1999. Comparative accuracies of artificial neural networks and discriminant analysis in predicting forest cover types from cartographic variables. *Computers and Electronics in Agriculture* 24 (1999), 131–151.

[4] Andrew Brock, Jeff Donahue, and Karen Simonyan. 2019. Large scale GAN training for high fidelity natural image synthesis. In *International Conference on Learning Representations (ICLR '19)*.

[5] Alberto Cano and Bartosz Krawczyk. 2020. Kappa updated ensemble for drifting data stream mining. *Machine Learning* 109 (01 2020), 175–218.

[6] Elliot Creager, Jörn-Henrik Jacobsen, and Richard Zemel. 2021. Environment inference for invariant learning. In *International Conference on Machine Learning*. PMLR, 2189–2200.

[7] Gabriela Csurka. 2017. *Domain Adaptation in Computer Vision Applications*. Springer.

[8] Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Ales Leonardis, Gregory Slabaugh, and Tinne Tuytelaars. 2022. A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 7 (2022), 3366–3385.

[9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 4171–4186.

[10] Ryan Elwell and Robi Polikar. 2011. Incremental learning of concept drift in nonstationary environments. *IEEE Trans. Neural Networks* 22, 10 (2011), 1517–1531.

[11] Wei Fan. 2004. Systematic data selection to mine concept-drifting data streams. In *Proceedings of the Tenth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 128–137.

[12] Jordi Fonollosa, Irene Rodríguez-Luján, and Ramón Huerta. 2015. Chemical gas sensor array dataset. *Data in Brief* 3 (2015), 85–89.

[13] João Gama, Pedro Medas, Gladys Castillo, and Pedro Rodrigues. 2004. Learning with drift detection. *Intelligent Data Analysis* 8, 286–295.

[14] Joáo Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. 2014. A survey on concept drift adaptation. *ACM Comput. Surv.* 46, 4 (2014), 1–37.

[15] João Gama, Indrundefined Žliobaitundefined, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. 2014. A survey on concept drift adaptation. *ACM Comput. Surv.* 46, 4, Article 44 (2014).

[16] Yaroslav Ganin, E. Ustinova, Hana Ajakan, Pascal Germain, H. Larochelle, François Laviolette, M. Marchand, and V. Lempitsky. 2016. Domain-adversarial training of neural networks. *J. Mach. Learn. Res.* 17 (2016), 59:1–59:35.

[17] Heitor Murilo Gomes, Jean Paul Barddal, Fabrício Enembreck, and Albert Bifet. 2017. A survey on ensemble learning for data stream classification. *ACM Comput. Surv.* 50, 2 (Mar. 2017), 1–36.

[18] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. Generative adversarial Networks. In *Advances in Neural Information Processing Systems (NeurIPS '14)*.

[19] Ruocheng Guo, Pengchuan Zhang, Hao Liu, and Emre Kiciman. 2021. Out-of-distribution prediction with invariant risk minimization: The limitation and an effective fix. *arXiv preprint arXiv:2101.07732* (2021).

[20] Xianyan Jia, Shutao Song, Wei He, Yangzihao Wang, Haidong Rong, Feihu Zhou, Liqiang Xie, Zhenyu Guo, Yuanzhou Yang, Liwei Yu, Tiegang Chen, Guangxiao Hu, Shaohuai Shi, and Xiaowen Chu. 2018. Highly scalable deep learning training system with mixed-precision: Training ImageNet in four minutes. *arXiv preprint arXiv:1807.11205* (2018).

[21] Tero Karras, Timo Aila, Samuli Laine, and Jaakko Lehtinen. 2018. Progressive growing of GANs for improved quality, stability, and variation. In *International Conference on Learning Representations (ICLR '18)*.

[22] Tero Karras, S. Laine, and Timo Aila. 2019. A style-based generator architecture for generative adversarial networks. *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2019), 4396–4405.

[23] Ioannis Katakis, Grigorios Tsoumakas, and I. Vlahavas. 2010. Tracking recurring contexts using ensemble classifiers: An application to email filtering. *Knowledge and Information Systems* 22 (03 2010), 371–391.

[24] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR '15)*.

[25] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. 2017. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences* 114, 13 (2017), 3521–3526.

[26] David Krueger, Ethan Caballero, Joern-Henrik Jacobsen, Amy Zhang, Jonathan Binas, Dinghuai Zhang, Remi Le Priol, and Aaron Courville. 2021. Out-of-distribution generalization via risk extrapolation (rex). In *International Conference on Machine Learning*. PMLR, 5815–5826.

[27] Ananya Kumar, Tengyu Ma, and Percy Liang. 2020. Understanding self-training for gradual domain adaptation. In *Proceedings of the 37th International Conference on Machine Learning*. 5468–5479.

[28] Sen Lin, Li Yang, Deliang Fan, and Junshan Zhang. 2022. TRGP: Trust region gradient projection for continual learning. In *International Conference on Learning Representations*.

[29] David Lopez-Paz and Marc' Aurelio Ranzato. 2017. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*.

[30] Jie Lu, Anjin Liu, Fan Dong, Feng Gu, Joáo Gama, and Guangquan Zhang. 2018. Learning under concept drift: A review. *IEEE Trans. Knowl. Data Eng.* 31, 12 (Oct. 2018), 2346–2363.
[31] Dominic Masters and Carlo Luschi. 2018. Revisiting small batch training for deep neural networks. *arXiv preprint arXiv:1804.07612* (2018).
[32] Jianmo Ni, Jiacheng Li, and Julian McAuley. 2019. Justifying recommendations using distantly-labeled reviews and fine-grained aspects. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP '19)*. 188–197.
[33] Vishal M. Patel, Raghuraman Gopalan, Ruonan Li, and Rama Chellappa. 2015. Visual domain adaptation: A survey of recent advances. *IEEE Signal Process. Mag.* 32, 3 (Apr. 2015), 53–69.
[34] Amanda Rios and Laurent Itti. 2019. Closed-loop memory GAN for continual learning. 3332–3338.
[35] Mohammad Rostami and Aram Galstyan. 2023. Overcoming concept shift in domain-aware settings through consolidated internal distributions. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 9623–9631.
[36] Kuniaki Saito, Kohei Watanabe, Y. Ushiku, and T. Harada. 2018. Maximum classifier discrepancy for unsupervised domain adaptation. *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2018), 3723–3732.
[37] Yujun Shen, Ceyuan Yang, Xiaoou Tang, and Bolei Zhou. 2020. InterFaceGAN: Interpreting the disentangled face representation learned by GANs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* PP (2020).
[38] Yujun Shen and Bolei Zhou. 2021. Closed-form factorization of latent semantics in GANs. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR '21)*.
[39] Hanul Shin, Jung Lee, Jaehong Kim, and Jiwon Kim. 2017. Continual learning with deep generative replay. (05 2017).
[40] Rui Shu, Hung Bui, Hirokazu Narui, and Stefano Ermon. 2018. A DIRT-T approach to unsupervised domain adaptation. (02 2018).
[41] Yiliao Song, Jie Lu, Anjin Liu, Haiyan Lu, and Guangquan Zhang. 2021. A segment-based drift adaptation method for data streams. *IEEE Transactions on Neural Networks and Learning Systems* (2021).
[42] Yiliao Song, Jie Lu, Haiyan Lu, and Guangquan Zhang. 2021. Learning data streams with changing distributions and temporal dependency. *IEEE Transactions on Neural Networks and Learning Systems* (2021).
[43] W. Nick Street and YongSeog Kim. 2001. A streaming ensemble algorithm (SEA) for large-scale classification. In *Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 377–382.
[44] Rishabh Tiwari, Krishnateja Killamsetty, Rishabh Iyer, and Pradeep Shenoy. 2022. GCR: Gradient coreset based replay buffer selection for continual learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 99–108.
[45] Alexander Vergara, Shankar Vembu, Tuba Ayhan, Margaret A. Ryan, Margie L. Homer, and Ramón Huerta. 2012. Chemical gas sensor drift compensation using classifier ensembles. *Sensors and Actuators B: Chemical* 166-167 (2012), 320–329.
[46] Haixun Wang, Wei Fan, Philip S. Yu, and Jiawei Han. 2003. Mining concept-drifting data streams using ensemble classifiers. In *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 226–235.
[47] Mei Wang and Weihong Deng. 2018. Deep visual domain adaptation: A survey. *Neurocomputing* 312 (2018), 135–153.
[48] Scott Wares, John Isaacs, and Eyad Elyan. 2019. Data stream mining: Methods and challenges for handling concept drift. *SN Appl. Sci.* 1, 11 (2019), 1–19.
[49] Gerhard Widmer and Miroslav Kubat. 1996. Learning in the presence of concept drift and hidden contexts. *Mach. Learn.* 23, 1 (Apr. 1996), 69–101.
[50] Lei Xu, Maria Skoularidou, Alfredo Cuesta-Infante, and Kalyan Veeramachaneni. 2019. Modeling tabular data using conditional GAN. In *Advances in Neural Information Processing Systems (NeurIPS '19)*.
[51] Marvin Mengxin Zhang, Henrik Marklund, Nikita Dhawan, Abhishek Gupta, Sergey Levine, and Chelsea Finn. 2021. Adaptive risk minimization: Learning to adapt to domain shift. In *Advances in Neural Information Processing Systems*.
[52] Zhifei Zhang, Yang Song, and Hairong Qi. 2017. Age progression/regression by conditional adversarial autoencoder. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR '17)*.

Received 18 April 2023; revised 23 January 2024; accepted 28 February 2024